

Как сделать правильный бэкап баз данных в PostgreSQL ?

0) **Так делать запрещено!**

Если просто скопировать папку PostgreSQL17/Databases/home/ - это приведёт к повреждению базы.

Нельзя копировать каталог данных при работающем сервере. Для работающей базы используются специальные утилиты.

1) **Способ №1 – через pg_dump (рекомендуемый).**

Создаём бэкап конкретной базы в custom-формате.

Это самый безопасный и правильный вариант.

Открываем cmd от админа и пишем:

```
cd "C:\Program Files\PostgreSQL\17\bin"
```

Таким образом, мы переходим в папку bin

```
pg_dump -U postgres -F c -d home -f C:\home.backup
```

либо же почти тоже самое, но с явным указанием хоста и порта:

```
pg_dump -U postgres -h 127.0.0.1 -p 5432 -F c -d home -f C:\home.backup
```

postgres – имя пользователя.

home – имя базы.

C:\home.backup – куда, под каким именем и в каком формате будет сделан бэкап.

После нажатия на Enter, он затребует пароль от админа Postgre, надеюсь вы его не забыли 😊

2) **Способ №2 – через pgAdmin, то есть через ГУИ.**

Для нормального боевого прода лучше использовать CLI и автоматизацию.

2.1 Открываем pgAdmin.

2.2 Находим нашу базу данных.

2.3 ПКМ по ней – backup.

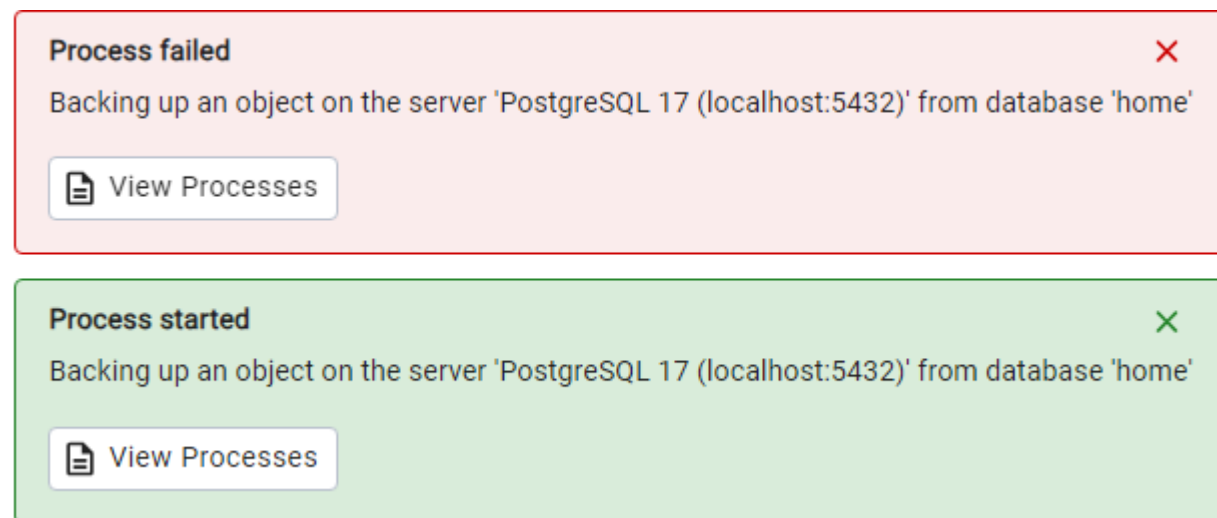
2.4 Формат должен быть Custom.

2.5 В поле Filename (он будет подсвечен красным) указываем путь и имя на английском.

2.6 Нажимаем «Backup» (значок с дискеткой) внизу справа.

Данный способ не самый эффективный и может не сработать.

После сохранения бэкапа, обязательно проверяйте его размер. Он никак не может весить 0 КБ!!!



3) **Бэкап ролей, паролей, прав и tablespaces.**

```
pg_dumpall -U postgres -h 127.0.0.1 -p 5432 --globals-only -f C:\roles.sql
```

Здесь тоже затребует пароль от админа Postgre. Вводим его, нажимаем Enter.

4) Как потом восстановить?

Здесь важен порядок команд!

Открываем cmd от админа и пишем:

```
# Восстанавливаем роли и права.  
psql -U postgres -f C:\roles.sql  
  
# Создаём пустую базу.  
  
# Если база home уже существует, сначала её нужно удалить вот такой командой:  
# dropdb -U postgres home  
createdb -U postgres home  
  
# Восстанавливаем содержимое базы из нашего файла с бэкапом.  
# pg_restore не создаёт базу автоматически, поэтому её нужно создать предыдущей командой.  
# Он просто загружает таблицы, индексы, последовательности и данные.  
# Флаг -c дополнительно очищает существующие объекты перед восстановлением.  
pg_restore -U postgres -d home -c C:\home.backup
```

5) Как всё это автоматизировать?

5.1 Открываем блокнот, создаём .bat-ник вот такого содержания:

```
@echo off  
setlocal  
  
REM --- Переходим в папку с утилитами PostgreSQL  
cd "C:\Program Files\PostgreSQL\17\bin"
```

REM --- Дата для имени файла

set DATE=%DATE:~6,4%-%DATE:~3,2%-%DATE:~0,2%

REM --- Папка для бэкапов

set BACKUP_DIR=C:\backup

REM --- Если папка не существует — создаём её

if not exist "%BACKUP_DIR%" mkdir "%BACKUP_DIR%"

REM --- Лог-файл

set LOGFILE=%BACKUP_DIR%\backup_%DATE%.log

echo ===== Backup started %DATE% %TIME% ===== >> "%LOGFILE%"

REM --- Бэкап ролей

pg_dumpall -U postgres -h 127.0.0.1 -p 5432 --globals-only -f "%BACKUP_DIR%\roles_%DATE%.sql" >> "%LOGFILE%" 2>&1

if %ERRORLEVEL% neq 0 (

echo ERROR: pg_dumpall failed! >> "%LOGFILE%"

exit /b 1

)

```
REM --- Бэкап базы home
pg_dump -U postgres -h 127.0.0.1 -p 5432 -F c -d home -f "%BACKUP_DIR%\home_%DATE%.backup" >> "%LOGFILE%" 2>&1
if %ERRORLEVEL% neq 0 (
    echo ERROR: pg_dump failed! >> "%LOGFILE%"
    exit /b 1
)

echo ===== Backup finished successfully %TIME% ===== >> "%LOGFILE%"
```

Данный .bat-файл содержит логирование, проверку %ERRORLEVEL%, выход с кодом ошибки и понятный лог-файл.

Если что-то упадёт – мы об этом узнаем. Плюс, папка для бэкапов создаётся автоматически.

5.2

Сохраняем файл как `C:\backup\backup_postgres.bat`

Папку `C:\backup` можно подготовить заранее, но на основании нашего скрипта, она будет создана автоматически. В ней мы будем хранить бэкапы баз данных.

5.3

Настраиваем запуск через планировщик задач Windows.

Win + R – `taskschd.msc`

Создать задачу... (Create Task...)

Вкладка Общие (самая первая) – даём имя, например, PostgreSQL Backup

Вкладка Триггеры (Triggers) - Создать (new)

Начало задачи: по расписанию (Daily)

Создание триггера

Начать задачу: По расписанию

Параметры

☐ Однократно

☐ Ежедневно

☒ Еженедельно

☐ Ежемесячно

Начать: 02.03.2026 15:00:00 ☐ Синхр. по поясам

Повторять каждую 1 неделю в:

☐ воскресенье ☒ понедельник ☒ вторник ☒ среда

☒ четверг ☒ пятница ☐ суббота

Вкладка Действия (Actions) – Создать (New)

Действие: запуск программы (start a program)

Программа или сценарий: указываем путь к нашему .bat-нику

Создание действия

Укажите действие для данной задачи.

Действие: Запуск программы

Параметры

Программа или сценарий:

C:\backup\backup_postgres.bat Обзор...

Добавить аргументы (необязательно):

Рабочая папка (необязательно):

OK Отмена

Вкладки Условия (Conditions) и Параметры (Settings) оставляем по умолчанию.

Сохраняем задачу. Готово!

Что мы получаем на выходе?

Каждый рабочий день, в 15:00 автоматически создаются:

```
C:\backup\roles_2026-03-02.sql
```

```
C:\backup\home_2026-03-02.backup
```

Эти файлы содержат все роли и актуальный дамп базы home.

Они будут разделены по датам и будет видно – от какой даты какой дамп.

Формат дампа .sql – для ролей, .backup – для базы данных.

Где можно посмотреть запущенные, или отключённые задачи в планировщике задач Windows?

Win + R – taskschd.msc

Библиотека планировщика заданий (вторая вкладка слева вверху).

6) Теперь сделаем файл, в котором бы хранился пароль для подключения к PostgreSQL, чтобы утилиты типа pg_dump, pg_restore, psql, pg_dumpall могли подключаться без запроса пароля вручную.

Для чего это нужно?

Чтобы команды работали:

[V] автоматически,

[V] без ввода пароля,

[V] через планировщик задач,

[V] в .bat-файлах / скриптах,

[V] на сервере / без интерактивной сессии.

Без этого файла, утилита просто будет ждать ввода пароля. Проще говоря – просто зависнет.

Итак, поехали!

Переходим по пути:

`C:\Users\имя_нашего_боевого_пользователя\AppData\Roaming\`

Здесь создаём папку `postgresql` и в ней создаём файл `pgpass.conf`

То есть, файл `pgpass.conf` должен находиться в профиле того пользователя, от которого запускается задача в планировщике!

Внутри файла записываем вот такой конфиг:

`127.0.0.1:5432:*:postgres:нашпарольотпользователяБД`

Сохраняем файл, закрываем, переходим в его свойства – вкладка «Безопасность».

Убрать доступ у «Все», оставить только у текущего пользователя.

Это нужно потому что PostgreSQL проверяет права.

Если файл доступен другим пользователям – он может игнорироваться.